



JavaScript中级-BOM

JSBOM

001.浏览器对象模型BOM:

BOM允许使用js来和浏览器进行交互

window对象:

windos代表的是浏览器的窗口，理解为HTML整个页面

所有全局的JS对象、函数、变量都会自动成为window的成员

全局变量是window的属性、全局函数是window的方法

文档对象DOM也是window的属性:

```
window.document.getElementById ("name")
```

等同于

```
document.fetElementById ("name")
```

窗口大小:

使用如下两个属性来确定浏览器窗口大小：不包括工具栏和滚动条

window.innerHeight，返回浏览器内部高度（像素）

window.innerWidth，返回浏览器内部宽度（像素）

```
let w = window.innerWidth;
```

```
let h = window.innerHeight;
```

其他窗口方法:

window.open ()，打开新窗口

window.close ()，关闭当前窗口

window.moveTo ()，移动当前窗口

window.resizeTo ()，调整当前窗口大小

002.窗口屏幕：

窗口可以不使用window前缀书写，具有以下属性：

- screen.width
- screen.height
- screen.availWidth
- screen.availHeight
- screen.colorDepth
- screen.pixelDepth

screen.width： 返回访问者屏幕的宽度，像素

```
document.getElementById("demo").innerHTML =  
"Screen Width: " + screen.width;
```

screen.height： 返回访问者屏幕的高度，像素

```
document.getElementById("demo").innerHTML =  
"Screen Height: " + screen.height;
```

screen.availWidth： 返回窗口可用宽度，即宽度减去任务栏等界面功能

```
document.getElementById ("demo") .innerHTML =  
"Available Screen Width:" + screen.availWidth;
```

screen.availHeight： 返回窗口可用高度，即高度减去任务栏等界面功能

```
document.getElementById("demo").innerHTML =  
"Available Screen Height: " + screen.availHeight;
```

screen.colorDepth： 返回一种颜色所用的位数

- 24 位 = 16,777,216 种不同的“真彩色”
- 32 位 = 4,294,967,296 种不同的“深色”

HTML 中使用的 #rrggbb (rgb) 值代表“真彩色”(16,777,216 种不同的颜色)

```
document.getElementById("demo").innerHTML =
```

```
"Screen Color Depth: " + screen.colorDepth;
```

screen.pixelDepth: 返回屏幕的像素深度

```
document.getElementById("demo").innerHTML =
```

```
"Screen Pixel Depth: " + screen.pixelDepth;
```

对于现代计算机来说，色彩深度和像素深度是相等的。

003.窗口位置

window.location，可以获取当前页面的地址URL，并将浏览器重新定向到新页面，可以忽略窗口前置**window**

- **window.location.href**返回当前页面的 href (URL)

```
document.getElementById("demo").innerHTML =
```

```
"Page location is " + window.location.href;
```

- **window.location.hostname**返回网络主机的域名

```
document.getElementById("demo").innerHTML =
```

```
"Page hostname is " + window.location.hostname;
```

- **window.location.pathname**返回当前页面的路径和文件名

```
document.getElementById("demo").innerHTML =
```

```
"Page path is " + window.location.pathname;
```

- **window.location.protocol**返回所使用的网络协议 (http: 或 https:)

```
document.getElementById("demo").innerHTML =
```

```
"Page protocol is " + window.location.protocol;
```

- **window.location.assign()**加载新文档，通过事件来触发函数

```
function newDoc() {
```

```
window.location.assign("https://www.w3schools.com")
```

```
}
```

```
</script>
```

- **window.location.port**返回互联网主机端口号，大多数浏览器不会显示默认端口号，**http: 80、https: 443**

```
document.getElementById ("demo") .innerHTML =
```

"Port number is " + window.location.port

004.窗口历史记录:

window.history: 包含浏览器的历史记录, 可以忽略窗口前缀window

history.back (): 与浏览器单击返回相同, 使用该方法加载历史列表的前一个URL

```
function goBack() {  
    window.history.back()  
}
```

history.forward (): 与浏览器单击前进相同, 加载历史列表的下一个URL, 使用事件触发。

```
function goForward() {  
    window.history.forward()  
}
```

005.窗口导航器:

导航器对象: navigator或window.navigator用于定位浏览器对象

浏览器cookie: 如果浏览器启用cookies, cookieEnabled属性返回true

```
document.getElementById("demo").innerHTML =  
"cookiesEnabled is " + navigator.cookieEnabled;
```

浏览器语言: language属性返回浏览器的语言

```
document.getElementById("demo").innerHTML = navigator.language;
```

浏览器是否在线: 浏览器在线, onLine属性返回true

```
document.getElementById("demo").innerHTML = navigator.onLine;
```

006.弹出框: alert、confirm、prompt

警告框: window.alert("sometext");可省略窗口前缀

```
alert("I am an alert box!");
```

确认框: window.confirm("sometext");

```
if (confirm("Press a button!")) {
```

```
txt = "You pressed OK!";  
} else {  
txt = "You pressed Cancel!";  
}
```

提示框：`window.prompt("sometext","defaultText");`

```
let person = prompt("Please enter your name", "Harry Potter");  
let text;  
if (person == null || person == "") {  
text = "User cancelled the prompt.";  
} else {  
text = "Hello " + person + "! How are you today?";  
}
```

换行符：反斜杠\n

```
alert("Hello\nHow are you?");
```

007.计时事件：

指js按时间间隔执行代码，主要方法是setTimeout、setInterval

setTimeout (function, milliseconds)：等待指定的毫秒数后执行函数

三秒后，页面提示Hello

```
<button onclick="myVar = setTimeout(myFunction, 3000)">Try it</button>  
<button onclick="clearTimeout(myVar)">Stop it</button>  
<script>  
function myFunction() {  
alert('Hello');  
}  
</script>
```

clearTimeout (settimeout函数)：用于停止计时函数的代码

setInterval (function, milliseconds)：与setTimeout相同，但是不断重复执行该函数

每一秒显示更新一次时间

```
setInterval(myTimer, 1000);  
function myTimer() {  
  const d = new Date();  
  document.getElementById("demo").innerHTML = d.toLocaleTimeString();  
}
```

clearInterval (setInterval函数)：用于停止计时函数代码

```
<button onclick="clearInterval(myVar)">Stop time</button>
```

008.cookies：

cookie：指存储与计算机上的小型文本文件的数据

- 当用户访问网页时，他/她的名字可以被存储在 cookie 中。
- 当用户下次访问该页面时，该 cookie 会“记住”他/她的名字。

cookie以名称-值的形式保存

```
username = John Doe
```

cookie具有document.cookie属性

JS创建cookie：默认情况下浏览器关闭会删除cookie

```
document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00  
UTC; path=/";
```

创建cookie时可选多个参数，UTC时间、cookie存储路径等，使用分号分隔

JS读取cookie：以字符串形式返回所有的cookie，类似cookie1=value;等

```
let x = document.cookie;
```

JS更改cookie：按照创建新的cookie来覆盖旧的cookie

```
document.cookie = "username=John Smith; expires=Thu, 18 Dec 2013  
12:00:00 UTC; path=/";
```

JS删除cookie：不用删除cookie值，只需要将cookie的参数设置为过去时间即可，要求cookie的路径准确。

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC;  
path=/";
```

cookie字符串：

将整个cookie字符串写入document.cookie中，当你再次读出它时，也只能看到它的名称-值对。

如果设置了新的 cookie2，旧的 cookie1 不会被覆盖。新的 cookie2 被添加到 document.cookie 中，因此如果再次读取 document.cookie，您将得到类似以下内容：

```
cookie1 = 值; cookie2 = 值;
```

cookie示例：创建一个存储访问者姓名的cookie，首次访问会要求填写姓名，下次访问会受到欢迎信息。

设置cookie：创建一个函数，将访问者姓名存储与cookie变量中

```
function setCookie(cname, cvalue, exdays) {  
  const d = new Date();  
  d.setTime(d.getTime() + (exdays*24*60*60*1000));  
  let expires = "expires=" + d.toUTCString();  
  document.cookie = cname + "=" + cvalue + ";" + expires + ";path=/";  
}
```

获取cookie：创建函数来返回制定cookie的值

```
function getCookie(cname) {  
  let name = cname + "=";  
  let decodedCookie = decodeURIComponent(document.cookie);  
  let ca = decodedCookie.split(';');  
  for(let i = 0; i <ca.length; i++) {  
    let c = ca[i];  
    while (c.charAt(0) == ' ') {  
      c = c.substring(1);  
    }  
    if (c.indexOf(name) == 0) {  
      return c.substring(name.length, c.length);  
    }  
  }  
}
```

```
return "";  
}
```

检查cookie：如果设置了cookie会显示问候语，反之要求输入用户名，并使用**setCookie**函数来存储用户名365天

```
function checkCookie() {  
    let username = getCookie("username");  
    if (username != "") {  
        alert("Welcome again " + username);  
    } else {  
        username = prompt("Please enter your name:", "");  
        if (username != "" && username != null) {  
            setCookie("username", username, 365);  
        }  
    }  
}
```